

ECRYP Project Report

Miller-Rabin Primality Test

Vianney CHAILLOU

Table des matières

1. Short Theory	2
1.1 Motivation	2
1.2 Fermat's Little Theorem and Its Weakness	2
1.3 The Miller-Rabin Improvement	2
1.4 Strong Pseudoprimes and the Security Parameter t	2
2. Functional Description.....	3
2.1 Input Data.....	3
2.2 Console Output Examples.....	3
2.3 Output Format	3
3. Code Structure	4
Block 1: <i>decompose(n_minus_1)</i>	4
Block 2: <i>mod_pow(base, exp, mod)</i>	4
Block 3: <i>millar_rabin_witness(n, a, s, r)</i>	4
Block 4: <i>millar_rabin(n, t=20)</i>	5
Block 5: <i>millar_rabin_deterministic(n)</i>	5
Block 6: <i>run_tests()</i>	5
Block 7: <i>interactive_mode()</i> and <i>main()</i>	5
4. Tests and Reference Values.....	6
4.1 Source of Reference Values	6
4.2 Test Cases Overview.....	6
4.3 Full Test Run	9
5. Conclusion.....	10

1. Short Theory

1.1 Motivation

A primality test decides whether a given positive integer n is prime or composite. Naive trial division is correct but becomes exponentially slow for large numbers. Efficient probabilistic tests are therefore essential in modern cryptography key generation for RSA or DSA that depend on finding large primes quickly.

1.2 Fermat's Little Theorem and Its Weakness

The Fermat primality test is based on Fermat's Little **Theorem 1**: if p is prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$.

The key weakness is that certain composite numbers called Carmichael numbers satisfy this congruence for every base a coprime to them. The smallest Carmichael number is $561 = 3 \times 11 \times 17$. Because of this, the Fermat test can never correctly classify Carmichael numbers as composite.

1.3 The Miller-Rabin Improvement

The Miller-Rabin test strengthens Fermat's criterion. For an odd n , write:

$$n - 1 = 2^s \cdot r$$

Where r is odd and $s \geq 1$

Theorem 2: if p is an odd prime and $\gcd(a, p) = 1$, then at least one of the following holds:

- $a^r \equiv 1 \pmod{p}$, OR
- $a^{2^j \cdot r} \equiv -1 \pmod{p}$ for some j in $\{0, \dots, s-1\}$

If neither condition holds for a given base a , then a is called a strong witness to compositeness for n , and n is definitely composite.

1.4 Strong Pseudoprimes and the Security Parameter t

An odd composite n that satisfies both conditions above for a particular base a is called a strong pseudoprime to base a . It is mathematically proven that, for any odd composite n , at most $\frac{1}{4}$ of all bases a in $[2, n-2]$ are strong liars. Unlike Fermat pseudoprimes, there is no analogue of Carmichael numbers for Miller-Rabin.

The algorithm repeats the test with t independently and uniformly chosen random bases. The probability that a composite number passes all t rounds is at most $(\frac{1}{4})^t$. For $t = 20$, this is less than 10^{-12} negligible in practice.

2. Functional Description

2.1 Input Data

The algorithm takes two main parameters:

- **n (integer):** This is the number to be tested for primality. It must be greater or equal than 2.
- **t (integer):** This is the security parameter, representing the number of independent rounds. It must be greater than or equal to 1 (in my code, it is set to 20 by default).

In interactive mode, both values are entered via the keyboard. In the automated test suite, they are hard-coded in the `run_tests()` function.

2.2 Console Output Examples

Composite number (Carmichael number 561):

```
Enter a positive integer to test (or 'q' to quit): 561
Enter security parameter t (number of rounds, default=20): 20

n = 561, t = 20
Result: COMPOSITE (certain)
```

Prime number:

```
Enter a positive integer to test (or 'q' to quit): 104729
Enter security parameter t (number of rounds, default=20): 20

n = 104729, t = 20
Result: PRIME (probable)
Note: probability of error <= (1/4)^20 ≈ 9.09e-13
```

2.3 Output Format

- **COMPOSITE (certain):** the algorithm found at least one strong witness to compositeness.
- **PRIME (probable):** all t rounds passed; error probability $\leq (\frac{1}{4})^t$.

3. Code Structure

The file *miller_rabin.py* is divided into 7 clearly labelled functional blocks.

Block 1: *decompose(n_minus_1)*

Purpose: Decompose the even integer $n - 1$ into $2^s \cdot r$ where r is odd.

- **Arguments:** *n_minus_1* (integer): The value $n - 1$, which must be an even number.
- **Returns:** tuple (s, r).

Method: repeatedly divide $n - 1$ by 2, counting iterations until an odd value remains.

Block 2: *mod_pow(base, exp, mod)*

Purpose: Compute $(base^{exp}) \bmod mod$ efficiently using the square-and-multiply (binary exponentiation) algorithm.

- **Arguments:**
 - *base* (integer): The base of the exponentiation.
 - *exp* (integer): The non-negative exponent.
 - *mod* (integer): The modulus.
- **Returns:** An integer representing the result.

Method: This function is critical: direct computation of a^{n-1} would produce astronomically large integers. By using the square-and-multiply method, all intermediate values are reduced modulo mod at each step, thereby avoiding exponential growth.

Block 3: *miller_rabin_witness(n, a, s, r)*

Purpose: Execute one round of the Miller-Rabin test for a single witness base a .

- **Arguments:**
 - n (integer): The number being tested.
 - a (integer): The witness base.
 - s (integer): Exponent of 2 in the decomposition of $n - 1$.
 - r (integer): Odd part of $n - 1$.
- **Returns:** True if a is a strong liar (n passes this round); False if a is a strong witness to compositeness.

Logic: compute $y = a^r \bmod n$, check if $y = 1$ or $y = n - 1$, then repeatedly square y up to $s - 1$ times looking for $n - 1$. If not found, return False.

Block 4: *miller_rabin(n, t=20)*

Purpose: Main probabilistic Miller-Rabin test. Runs t independent rounds with randomly chosen bases.

- **Arguments :**
 - n (integer): The integer to test for primality.
 - t (integer): Security parameter representing the number of independent rounds (default = 20).
- **Returns:** True (probably prime) or False (definitely composite).

Method: Handles trivial cases ($n < 2$, $n = 2$, $n = 3$, even n) directly before the main loop.

Block 5: *miller_rabin_deterministic(n)*

Purpose: Deterministic variant using the fixed witness set $\{2, 3, 5, 7\}$, valid for all $n < 3,215,031,751$ (a known mathematical threshold where the first four primes are sufficient as fixed bases).

- **Arguments:** n (integer): The integer to test.
- **Returns:** A boolean indicating if the number is prime (True) or composite (False).

Method: Runs the Miller-Rabin witness check using the first four primes as fixed bases instead of random ones.

Block 6: *run_tests()*

Purpose: Automated test suite with 7 carefully selected test cases.

Returns: Console output displaying the test execution and results.

Method: Each case compares the result of *miller_rabin()* with a reference value produced by a simple built-in trial-division oracle (*is_prime_reference()*), which requires no external library.

Block 7: *interactive_mode()* and *main()*

Purpose: *interactive_mode()* provides a console loop allowing the user to test arbitrary numbers. *main()* calls *run_tests()* first, then launches *interactive_mode()*.

Returns: Console interaction prompts and final results for the user.

Method: Loops user input from the keyboard, calling the primality test on the provided integers until the user decides to quit.

4. Tests and Reference Values

4.1 Source of Reference Values

Reference values are computed by the function `is_prime_reference(n)` defined inside `run_tests()`. This function uses trial division up to the square root of n a simple, deterministic, and universally trusted algorithm. It requires no external cryptographic library and is used only for comparison purposes.

4.2 Test Cases Overview

#	n	Description	Expected	Result
1	7	Small known prime	PRIME	PRIME
2	104729	Large known prime (10,001st prime)	PRIME	PRIME
3	100	Even composite number	COMPOSITE	COMPOSITE
4	91	Odd composite / Fermat pseudoprime base 3	COMPOSITE	COMPOSITE
5	561	Carmichael number (3 x 11 x 17)	COMPOSITE	COMPOSITE
6	1105	Carmichael number (5 x 13 x 17)	COMPOSITE	COMPOSITE
7	2047	Strong pseudoprime base 2 (23 x 89)	COMPOSITE	COMPOSITE

Test #01: Small known prime: $n = 7$

Why chosen: Smallest odd prime beyond 2 and 3. Validates that the algorithm handles trivial primes correctly and does not short-circuit them as composite.

```
Test #01 | n = 7
Description : Small known prime
Miller-Rabin result (t=20) : PRIME
Reference result (trial div): PRIME
Status : PASS
```

Test #02: Large known prime: $n = 104,729$

Why chosen: The 10,001st prime. Tests the algorithm on a large number where the modular exponentiation path is fully exercised, while trial division is still fast enough to produce a trusted reference result.

```
-----  
Test #02 | n = 104729  
Description : Large known prime (10001st prime)  
Miller-Rabin result (t=20) : PRIME  
Reference result (trial div): PRIME  
Status : PASS  
-----
```

Test #03: Even composite: $n = 100$

Why chosen: Trivially composite even number. The algorithm should short-circuit immediately on the even-check without entering the main loop, confirming correct early termination.

```
-----  
Test #03 | n = 100  
Description : Even composite number  
Miller-Rabin result (t=20) : COMPOSITE  
Reference result (trial div): COMPOSITE  
Status : PASS  
-----
```

Test #04: Fermat pseudoprime to base 3: $n = 91 = 7 \times 13$

Why chosen: 91 is a Fermat pseudoprime to base 3 ($3^{90} \equiv 1 \pmod{91}$), yet it is clearly composite. Miller-Rabin should detect it as composite, confirming it is stronger than the Fermat test.

```
-----  
Test #04 | n = 91  
Description : Odd composite - Fermat pseudoprime to base 3 (7 x 13)  
Miller-Rabin result (t=20) : COMPOSITE  
Reference result (trial div): COMPOSITE  
Status : PASS  
-----
```

Test #05: Carmichael number: $n = 561 = 3 \times 11 \times 17$

Why chosen: 561 is the smallest Carmichael number. A pure Fermat test would classify it as prime for every base a coprime to 561. This test proves that Miller-Rabin correctly detects it as composite, demonstrating its key advantage over Fermat. This is the most important test case.

```
-----  
Test #05 | n = 561  
Description : Carmichael number (3 × 11 × 17) – fools Fermat test  
Miller-Rabin result (t=20) : COMPOSITE  
Reference result (trial div): COMPOSITE  
Status : PASS  
-----
```

Test #06: Carmichael number: $n = 1105 = 5 \times 13 \times 17$

Why chosen: The second smallest Carmichael number. Same motivation as Test #05. This test reinforces that Miller-Rabin has no equivalent of Carmichael numbers: it is guaranteed that for any odd composite n , at most $\frac{1}{4}$ of all bases are strong liars.

```
-----  
Test #06 | n = 1105  
Description : Carmichael number (5 × 13 × 17) – fools Fermat test  
Miller-Rabin result (t=20) : COMPOSITE  
Reference result (trial div): COMPOSITE  
Status : PASS  
-----
```

Test #07: Strong pseudoprime to base 2: $n = 2047 = 23 \times 89$

Why chosen: 2047 is the smallest composite integer that is a strong pseudoprime to base 2. A single-base Miller-Rabin test with only base 2 would classify it as prime. Our probabilistic version with $t = 20$ independent random bases correctly identifies it as composite, illustrating exactly why multiple rounds are essential.

```
-----  
Test #07 | n = 2047  
Description : Strong pseudoprime to base 2 (23 × 89) – from delta_1 table  
Miller-Rabin result (t=20) : COMPOSITE  
Reference result (trial div): COMPOSITE  
Status : PASS  
-----
```


4.3 Full Test Run

```
=====
MILLER-RABIN PRIMALITY TEST - Automated Test Suite (7 cases)
ECRYP Project 2026 | Warsaw University of Technology
=====

Test #01 | n = 7
Description : Small known prime
Miller-Rabin result (t=20) : PRIME
Reference result (trial div): PRIME
Status : PASS
-----

Test #02 | n = 104729
Description : Large known prime (10001st prime)
Miller-Rabin result (t=20) : PRIME
Reference result (trial div): PRIME
Status : PASS
-----

Test #03 | n = 100
Description : Even composite number
Miller-Rabin result (t=20) : COMPOSITE
Reference result (trial div): COMPOSITE
Status : PASS
-----

Test #04 | n = 91
Description : Odd composite - Fermat pseudoprime to base 3 (7 x 13)
Miller-Rabin result (t=20) : COMPOSITE
Reference result (trial div): COMPOSITE
Status : PASS
-----

Test #05 | n = 561
Description : Carmichael number (3 x 11 x 17) - fools Fermat test
Miller-Rabin result (t=20) : COMPOSITE
Reference result (trial div): COMPOSITE
Status : PASS
-----

Test #06 | n = 1105
Description : Carmichael number (5 x 13 x 17) - fools Fermat test
Miller-Rabin result (t=20) : COMPOSITE
Reference result (trial div): COMPOSITE
Status : PASS
-----

Test #07 | n = 2047
Description : Strong pseudoprime to base 2 (23 x 89) - from delta_1 tab
Miller-Rabin result (t=20) : COMPOSITE
Reference result (trial div): COMPOSITE
Status : PASS
-----

Overall result: ALL TESTS PASSED
=====

=====
MILLER-RABIN PRIMALITY TEST - Interactive Mode
=====

Enter a positive integer to test (or 'q' to quit): █
```

5. Conclusion

The implemented Miller-Rabin algorithm correctly handles all 7 test cases, including the pathological cases that defeat the simpler Fermat test — Carmichael numbers ($n = 561, 1105$) and strong pseudoprimes ($n = 2047$). The code is structured in 7 clearly separated functional blocks, uses only built-in Python arithmetic for the core algorithm, and provides both an automated test suite and an interactive console interface.

The error probability for $t = 20$ rounds is bounded by $(\frac{1}{4})^{20} \approx 9,1 \cdot 10^{-13}$, making the test entirely suitable for cryptographic applications such as RSA key generation.